龙邱科技
LONGQIUKEJI

龙邱-龙芯 2K0300
开源库
使用介绍

—— 龙邱科技

目录

前言 1

一、刚拿到库时的配置方法 2

1.1 CMakeLists.txt 文件修改 2

1.2 程序编译 3

1.3 驱动模块 4

1.4 VScode 解决头文件报错问题 5

二、各模块测试函数的使用 6

三、GPIO 输入输出功能 7

2.1 设备文件实现 GPIO 输入输出功能 7

2.2 寄存器实现 GPIO 输入输出功能 8

四、PWM 控制器输出 PWM 8

3.1 设备文件实现 PWM 控制器输出 PWM 9

3.2 寄存器实现 PWM 控制器输出 PWM 10

五、寄存器配置 GTIM 控制器输出 PWM 10

六、寄存器配置 PWM 控制器实现编码器功能 11

七、电机功能测试 12

八、其他功能测试 12

九、相关文件与视频教程 13

前言

全国大学生智能车竞赛发展到今天，可以使用的芯片种类越来越多，每个芯片所使用的内核和底层硬件也是千差万别，而对于我们智能车选手而言，更关注的是上层的应用逻辑程序，如果把大量时间都花费到解决芯片底层硬件上，无疑会加重我们的负担，为此，龙邱科技为每一个智能车组别所使用的芯片都编写了对应的开源库。

龙邱的龙芯开源库采用模块化编程，把每个智能车所使用到的模块和外设都写了单独的.cpp 和.hpp 文件，学习单独模块只需要查看对应的文件即可。

我们将智能车比赛所用到的所有模块和外设都写了测试例程供大家使用，且函数调用都放到了 main()函数里面，用户想要测试哪个模块，直接将相应函数的注释去掉就可以直接测试。

每个模块和外设使用的引脚我们都通过传参的方式进行使用，在使用对应类的有参构造函数时，传入对应引脚号即可。

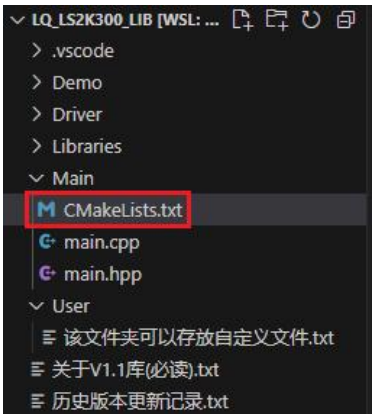
大家在使用库中类的时候，要注意对应类的功能情况，以及所对应的有哪些引脚。

一、刚拿到库时的配置方法

在将库拿到手后，是需要配置一些路径的，无法直接使用。

1.1 CMakeLists.txt 文件修改

首先找到 CMakeLists.txt 文件：



将交叉编译工具的路径修改为自己的存放路径：

将红框位置的路径修改为自己的 OpenCV 头文件以及库的存放路径：

虚拟机中的存放位置，CMakeLists.txt 文件中的填写路径以及久久派中的存

放位置都要一致。

虚拟机中：

```
(base) lqkj@DESKTOP-LQKJ:~/99_pai/ls2k0300/LQ_ls2k300_LIB$ ls /usr/local/include
opencv4
```

```
(base) lqkj@DESKTOP-LQKJ:~/99_pai/ls2k0300/LQ_ls2k300_LIB$ ls /usr/local/lib
cmake                          libopencv_features2d.so.410    libopencv_imgcodecs.so.410     libopencv_photo.so.410
libopencv_calib3d.so           libopencv_flann.so             libopencv_imgproc.so           libopencv_stitching.so
libopencv_calib3d.so.4.10.0    libopencv_flann.so.4.10.0      libopencv_imgproc.so.4.10.0    libopencv_stitching.so.4.10.0
libopencv_calib3d.so.410       libopencv_flann.so.410         libopencv_imgproc.so.410       libopencv_stitching.so.410
libopencv_core.so              libopencv_gapi.so              libopencv_ml.so                libopencv_video.so
libopencv_core.so.4.10.0       libopencv_gapi.so.4.10.0       libopencv_ml.so.4.10.0         libopencv_video.so.4.10.0
libopencv_core.so.410          libopencv_gapi.so.410          libopencv_ml.so.410            libopencv_video.so.410
libopencv_dnn.so               libopencv_highgui.so           libopencv_objdetect.so         libopencv_videoio.so
libopencv_dnn.so.4.10.0        libopencv_highgui.so.4.10.0    libopencv_objdetect.so.4.10.0  libopencv_videoio.so.4.10.0
libopencv_dnn.so.410           libopencv_highgui.so.410       libopencv_objdetect.so.410     libopencv_videoio.so.410
libopencv_features2d.so        libopencv_imgcodecs.so         libopencv_photo.so             python2.7
libopencv_features2d.so.4.10.0 libopencv_imgcodecs.so.4.10.0  libopencv_photo.so.4.10.0      python3.8
```

久久派中：

```
root@LoongOS:~# ls /usr/local/include/
opencv4
root@LoongOS:~# ls /usr/local/lib/
cmake                          libopencv_imgcodecs.so.410
libopencv_calib3d.so           libopencv_imgproc.so
libopencv_calib3d.so.4.10.0    libopencv_imgproc.so.4.10.0
libopencv_calib3d.so.410       libopencv_imgproc.so.410
libopencv_core.so              libopencv_ml.so
libopencv_core.so.4.10.0       libopencv_ml.so.4.10.0
libopencv_core.so.410          libopencv_ml.so.410
libopencv_dnn.so               libopencv_objdetect.so
libopencv_dnn.so.4.10.0        libopencv_objdetect.so.4.10.0
libopencv_dnn.so.410           libopencv_objdetect.so.410
libopencv_features2d.so        libopencv_photo.so
libopencv_features2d.so.4.10.0 libopencv_photo.so.4.10.0
libopencv_features2d.so.410    libopencv_photo.so.410
libopencv_flann.so             libopencv_stitching.so
libopencv_flann.so.4.10.0      libopencv_stitching.so.4.10.0
libopencv_flann.so.410         libopencv_stitching.so.410
libopencv_gapi.so              libopencv_video.so
libopencv_gapi.so.4.10.0       libopencv_video.so.4.10.0
libopencv_gapi.so.410          libopencv_video.so.410
libopencv_highgui.so           libopencv_videoio.so
libopencv_highgui.so.4.10.0    libopencv_videoio.so.4.10.0
libopencv_highgui.so.410       libopencv_videoio.so.410
libopencv_imgcodecs.so         python2.7
libopencv_imgcodecs.so.4.10.0  python3.8
```

若想要添加新的源文件路径，可用以下函数，其中 PROJECT_SOURCE_DIR 代表的是当前 CMakeLists.txt 文件所在的路径：

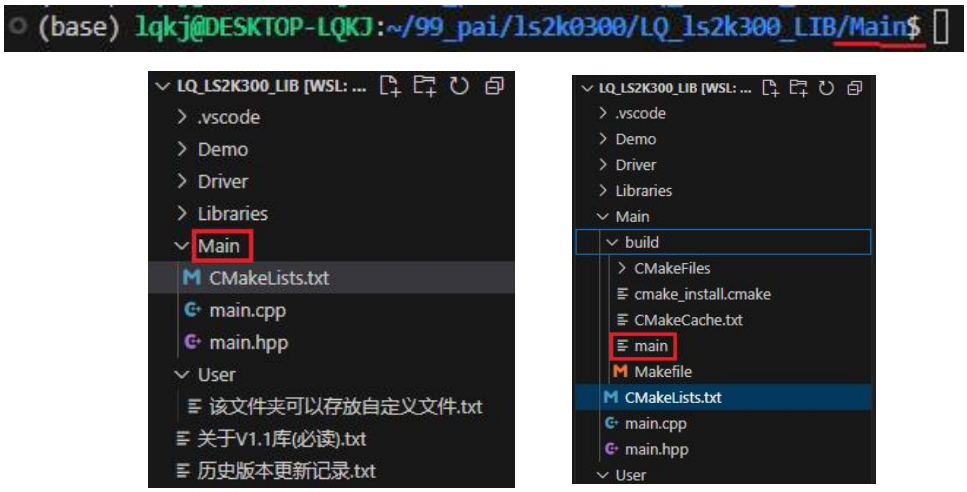
```
# 指定所用的源文件路径
aux_source_directory(${PROJECT_SOURCE_DIR}/ SRC)
```

若想要添加新的头文件路径，可用以下函数：

```
# 指定所用的头文件路径
include_directories(${PROJECT_SOURCE_DIR}/)
```

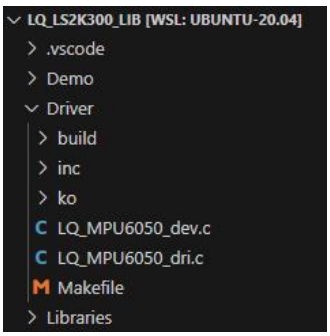
1.2 程序编译

在编译程序时，要去到与 CMakeLists.txt 文件所在路径下运行编译指令，生成好的可执行程序会存放在所生成的 build 目录下：

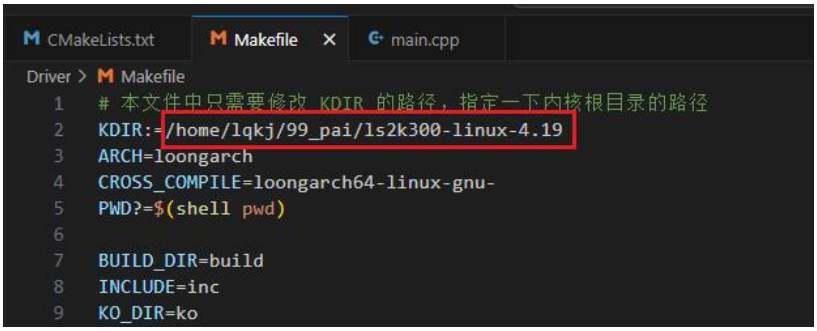


1.3 驱动模块

位置在 Driver 文件中，其中 build 文件夹存放编译时生成的中间文件，inc 文件夹存放使用的头文件，ko 文件夹存放编译生成出来的.ko 模块文件。

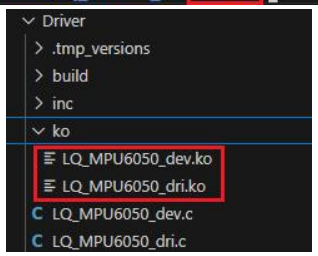


路径修改，需要将 Makefile 文件中的 KDIR 路径修改指定为自己的内核路径：



编译时，需要进入到 Driver 文件夹中，输入 make 指令进行编译：

```
(base) lqkj@DESKTOP-LQKJ:~/99_pai/ls2k0300/LQ_ls2k300_LIB/Driver$ make
make -C /home/lqkj/99_pai/ls2k300-linux-4.19 M=/home/lqkj/99_pai/ls2k0300/LQ_ls2k300_LIB/Driver modules
make[1]: Entering directory '/home/lqkj/99_pai/ls2k300-linux-4.19'
  CC [M]  /home/lqkj/99_pai/ls2k0300/LQ_ls2k300_LIB/Driver/LQ_MPU6050_dev.o
  CC [M]  /home/lqkj/99_pai/ls2k0300/LQ_ls2k300_LIB/Driver/LQ_MPU6050_dri.o
  Building modules, stage 2.
  MODPOST 2 modules
  CC      /home/lqkj/99_pai/ls2k0300/LQ_ls2k300_LIB/Driver/LQ_MPU6050_dev.mod.o
  LD [M]  /home/lqkj/99_pai/ls2k0300/LQ_ls2k300_LIB/Driver/LQ_MPU6050_dev.ko
  CC      /home/lqkj/99_pai/ls2k0300/LQ_ls2k300_LIB/Driver/LQ_MPU6050_dri.mod.o
  LD [M]  /home/lqkj/99_pai/ls2k0300/LQ_ls2k300_LIB/Driver/LQ_MPU6050_dri.ko
make[1]: Leaving directory '/home/lqkj/99_pai/ls2k300-linux-4.19'
(base) lqkj@DESKTOP-LQKJ:~/99_pai/ls2k0300/LQ_ls2k300_LIB/Driver$
```



想要清理所生成的文件，可以使用 make clean 指令进行清理：

```
(base) lqkj@DESKTOP-LQKJ:~/99_pai/ls2k0300/LQ_ls2k300_LIB/Driver$ make clean
rm build/* build/.LQ* ko/* .tmp* -rf
```

使用时，可以在终端使用 insmod 指令加载模块，rmmod 指令卸载模块，lsmod 指令查看模块：

```
root@LoongOS:test# insmod LQ_ls_i2c_dri.ko
root@LoongOS:test# insmod LQ_ls_i2c_dev.ko
[   96.006280] LQ_ls_i2c_dri module probe function
[   96.026406] LQ_ls_i2c_dev init function
root@LoongOS:test# lsmod
Module                  Size  Used by
LQ_ls_i2c_dev          65536  0
LQ_ls_i2c_dri          65536  0
aic8800_fdrv          704512  0
aic8800_bsp           163840  1 aic8800_fdrv
```

```
root@LoongOS:test# rmmod LQ_ls_i2c_dri.ko
[   106.025281] LQ_ls_i2c_dri module remove function
root@LoongOS:test# rmmod LQ_ls_i2c_dev.ko
[   109.623476] LQ_ls_i2c_dev exit function
root@LoongOS:test# lsmod
Module                  Size  Used by
aic8800_fdrv          704512  0
aic8800_bsp           163840  1 aic8800_fdrv
```

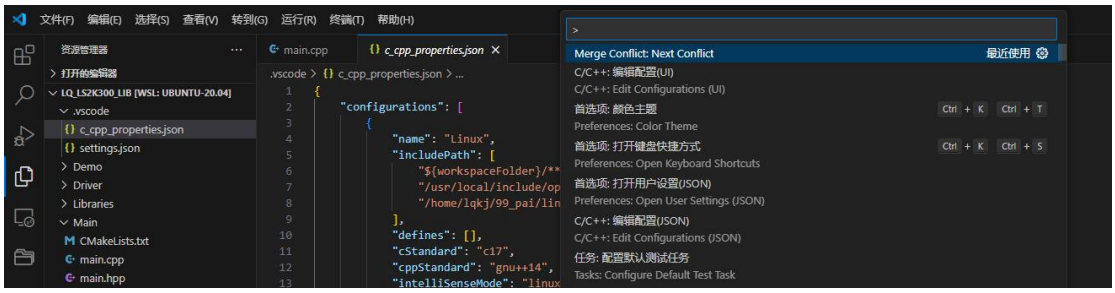
如果想要在程序中自动使用模块的话，可以尝试使用 system()函数，将需要用到的指令转换为字符串的形式填入就行，当然也可以带上路径：

```
system("insmod LQ_MPU6050_dri.ko");
```

```
system("insmod /root/LQ_MPU6050_dri.ko");
```

1.4 VScode 解决头文件报错问题

可以按下 Ctrl + Shift + p，在命令框搜索 Edit Configurations：



选择之后会生成.vscode 文件，在 c_cpp_properties.json 文件的 includePath 中填入需要指定的文件路径即可：

二、各模块测试函数的使用

在开源库的 main.cpp 文件中，我们在 main()函数里面放置了各个模块的测试函数用来方便大家的调试，测试哪个模块或功能直接去掉哪个函数的注释即可，不用再打开一个新的工程，在后期开发中可以复制一个工程用来备份，然后直接在本工程上开发自己的项目，不用再将各个模块移植到同一个工程中进行融合，使用起来非常方便。

```
// GpioOutputDemo1(88);     // 测试GPIO输出功能(设备文件)
// GpioOutputDemo2(88);     // 测试GPIO输出功能(寄存器)
// GpioInputDemo1();        // 测试GPIO输入功能(设备文件)
// GpioInputDemo2();        // 测试GPIO输入功能(寄存器)
// PwmDevDemo();            // 测试 PWM 功能(设备文件)
// PwmHWDemo();             // 测试 PWM 功能(寄存器)
// GtimPwmDemo();           // 测试 Gtim 的 PWM 功能(寄存器)
// EncoderDemo();           // 测试编码器功能(寄存器)
// CameraDemo();            // 测试摄像头功能
// AdcFunDemo();            // 测试 ADC 功能
// TFTDemo();               // 测试 TFT 屏幕功能
// Demo_IIC_Gyro();         // 测试陀螺仪功能
// Demo_I2C_Gyro();         // 测试陀螺仪驱动功能
// Uart_Demo();             // 测试串口功能
// GetTimeDemo();           // 测试时间戳打印功能
// sleepDemo();             // 测试 sleep() 函数 ------------ 以秒为单位延时
// usleepDemo();            // 测试 usleep() 函数 ----------- 以微秒为单位延时
// nanosleepDemo();         // 测试 nanosleep() 函数 -------- 以纳秒为单位延时
// clock_nanosleepDemo();   // 测试 clock_nanosleep() 函数 -- 以纳秒为单位延时
// MotorDemo();             // 测试电机功能
// ServoDemo();             // 测试舵机功能
// GpioDemo();              // 测试久久派22个GPIO翻转功能
```

如图，目前未开启测试程序，需要测试哪个，直接取消注释，编译运行即可。在拿到产品时，接线正确的情况下，直接运行测试程序即可检测产品的好坏。

注意：下面各函数中所使用到的宏，都定义在对应.hpp 中，可以在其中找到。

三、GPIO 输入输出功能

在龙芯 2K0300 中，大家可以使用设备文件或者配置寄存器的方式，实现 GPIO 的输入输出功能。

```
// GpioOutputDemo1(88);     // 测试GPIO输出功能(设备文件)
// GpioOutputDemo2(88);     // 测试GPIO输出功能(寄存器)
// GpioInputDemo1();        // 测试GPIO输入功能(设备文件)
// GpioInputDemo2();        // 测试GPIO输入功能(寄存器)
```

这四个测试函数，分别对应了 GPIO 输入输出功能的设备文件实现和寄存器实现，其中 GPIO 输出功能实现的函数中，填入的参数就是大家所使用的引脚号。

2.1 设备文件实现 GPIO 输入输出功能

```
main.cpp    LQ_GPIO.cpp ×
src > LQ_GPIO.cpp > SetGPIO(uint8_t, const string &)
33  /*LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL
34   * @函数名称: SetGPIO::SetGPIO(uint8_t gpio, const string &dir)
35   * @功能说明: GPIO 类的带参构造函数，配置 GPIO 并设置模式
36   * @参数说明: gpio  :  对应使用的引脚号
37   * @参数说明: dir   :  GPIO 模式
38   * @函数返回: 无
39   * @调用方法: SetGPIO MyGpio(88, OUT);
40   * @备注说明: 设置 GPIO 模式时，IN 为输入模式，OUT 为输出模式
41   QQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQ*/
42  SetGPIO::SetGPIO(uint8_t gpio, const string &dir)
```

该类，就是使用设备文件的方式配置 GPIO 的输入输出功能。其中有两个参数，参数一表示对应使用的引脚号；参数二表示对应的 GPIO 模式，设置为 IN 表示输入模式，设置为 OUT 表示输出模式。其他对应的函数以及函数使用说明注释，都可以在该文件下找到。使用该类时属于变量创建，可以在全局区中创建，

但创建完成之后，使用类中函数时，属于函数调用，不可以在全局区使用。

2.2 寄存器实现 GPIO 输入输出功能

```
main.cpp    LQ_HW_GPIO.cpp ×
src > LQ_HW_GPIO.cpp > HWGpio(uint8_t, uint8_t)
35
36  /*LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL
37   * @函数名称: HWGpio::HWGpio(uint8_t gpio, uint8_t mode) : Gpio(gpio)
38   * @功能说明: 硬件配置 GPIO 的有参构造函数
39   * @参数说明: gpio  :  对应使用的引脚号
40   * @参数说明: mode  :  GPIO 模式
41   * @函数返回: 无
42   * @调用方法: HWGpio MyGpio(88, GPIO_Mode_Out);
43   * @备注说明: 无
44   QQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQ*/
45  HWGpio::HWGpio(uint8_t gpio, uint8_t mode) : Gpio(gpio)
```

该类，就是使用寄存器的方式配置 GPIO 的输入输出功能。其中有两个参数，参数一表示对应使用的引脚号；参数二表示对应的 GPIO 模式，设置为 GPIO_Mode_In 表示输入模式，设置为 GPIO_Mode_Out 表示输出模式。其他对应的函数以及函数使用说明注释，都可以在该文件下找到。使用该类时属于变量创建，可以在全局区中创建，但创建完成之后，使用类中函数时，属于函数调用，不可以在全局区使用。

四、PWM 控制器输出 PWM

在龙芯 2K0300 中，大家可以使用设备文件或者配置寄存器的方式，实现 PWM 控制器输出 PWM 波。

```
// PwmDevDemo();          // 测试 PWM 功能(设备文件)
// PwmHWDemo();           // 测试 PWM 功能(寄存器)
```

这两个测试函数，分别对应了 PWM 控制器使用设备文件以及寄存器实现输出 PWM 的功能。

3.1 设备文件实现 PWM 控制器输出 PWM

```
main.cpp    LQ_PWM.cpp ×
src > LQ_PWM.cpp > SetPWM(const string &, int, int, const string &)
35  /*LLLL
```

3.2 寄存器实现 PWM 控制器输出 PWM

```
main.cpp    LQ_HW_PWM.cpp ×
src > LQ_HW_PWM.cpp > HWPwm(uint8_t, uint8_t, uint8_t, uint32_t, uint32_t, int)
32  /*LLLL
```

```
main.cpp    LQ_GTIM_PWM.cpp ×
src > LQ_GTIM_PWM.cpp > ...
/*LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL
 * @函数名称: GtimPwm::GtimPwm(uint8_t gpio, uint8_t chNum, uint8_t pol, uint32_t period, uint32_t duty_cycle, int mux)
                    : Gpio(gpio), Pol(pol), Period(period), Duty_cycle(duty_cycle), ChNum(chNum - 1)
 * @功能说明: GTIM 控制器 PWM 模式的有参构造函数
 * @参数说明: gpio        :  输出 PWM 的 GPIO 引脚号
 * @参数说明: chNum       :  参数一对应的通道号
 * @参数说明: pol         :  PWM 输出极性
 * @参数说明: period      :  PWM 周期
 * @参数说明: duty_cycle  :  PWM 占空比
 * @参数说明: mux         :  复用 GPIO 引脚
 * @函数返回: 无
 * @调用方法: GtimPwm MyPwm(88, 2, LS_GTIM_INVERSED, 2000000 - 1, 150000);
 * @备注说明: 参数六设置有默认值，可以不修改，GPIO 对应的通道号可查看:
 *            《龙芯2K0300处理器用户手册》
 QQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQ*/
GtimPwm::GtimPwm(uint8_t gpio, uint8_t chNum, uint8_t pol, uint32_t period, uint32_t duty_cycle, int mux)
    : Gpio(gpio), Pol(pol), Period(period), Duty_cycle(duty_cycle), ChNum(chNum - 1)
```

该类，就是使用寄存器的方式配置 GTIM 控制器输出 PWM。其中有六个参数，第一个参数表示输出 PWM 的引脚号；第二个参数表示使用的引脚对应的通道号；第三个参数表示设置的 PWM 极性；第四个参数表示设置的 PWM 周期，该类周期计算较为特殊，$周期 = \frac{10^8}{cycle}$；第五个参数表示设置的 PWM 占空比；第六个参数表示使用引脚的复用情况。其他对应的函数以及函数使用说明注释，都可以在该文件下找到。使用该类时属于变量创建，可以在全局区中创建，但创建完成之后，使用类中函数时，属于函数调用，不可以在全局区使用。

六、寄存器配置 PWM 控制器实现编码器功能

在龙芯 2K0300 中，大家可以使用配置寄存器的方式，通过 PWM 控制器实现编码器功能。

```
// EncoderDemo();          // 测试编码器功能(寄存器)
```

该测试函数就是配置 PWM 控制器实现对编码器的脉冲测量功能测试。

```
main.cpp    LQ_PWM_ENCODER.cpp ×
src > LQ_PWM_ENCODER.cpp > ...
/*LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL
 * @函数名称: LS_PwmEncoder::LS_PwmEncoder(int channel, int gpio) :
                DirGPIO(gpio, GPIO_Mode_In)
 * @功能说明: 配置编码器类的有参构造函数
 * @参数说明: channel  :  PWM 的通道编号
 * @参数说明: gpio     :  检测编码器方向的引脚编号
 * @函数返回: 无
 * @调用方法: encoder.Init();
 * @备注说明: 无
 QQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQ*/
LS_PwmEncoder::LS_PwmEncoder(int channel, int gpio) : DirGPIO(gpio, GPIO_Mode_In)
```

该类，就是使用寄存器的方式配置 PWM 控制器实现编码器脉冲检测功能。其中有两个参数，第一个参数表示编码器脉冲引脚所使用的 PWM 控制器的通道编号；第二个参数表示编码器方向引脚所使用的 GPIO 引脚号。其他对应的函数以及函数使用说明注释，都可以在该文件下找到。使用该类时属于变量创建，可以在全局区中创建，但创建完成之后，使用类中函数时，属于函数调用，不可以在全局区使用。

七、电机功能测试

```
/*LLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLLL
 * @函数名称: void MotorDemo()
 * @功能说明: 电机功能测试
 * @参数说明: 无
 * @函数返回: 无
 * @调用方法: MotorDemo();
 * @备注说明: 使电机每隔 2 秒转动方向翻转一次
 QQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQQ*/
void MotorDemo()
{
    /* 电机PWM */
    SetPWM LMotorPWM(PWM1, 100000, 10, "inversed"); // TIM2_CH1 GPIO87
    SetPWM RMotorPWM(PWM2, 100000, 10, "inversed"); // TIM2_CH2 GPIO88
    /* 电机方向GPIO */
    HWGpio LMotoDir(74, GPIO_Mode_Out); // CAN3_RX  left motor dir
    HWGpio RMotoDir(75, GPIO_Mode_Out); // CAN3_TX  right motor dir
    // 设置占空比
    LMotorPWM.SetDutyCycle(30000);
    RMotorPWM.SetDutyCycle(30000);
    // 使能
    LMotorPWM.Enable();
    RMotorPWM.Enable();

    while(true)
    {
        LMotoDir.SetGpioValue(1);
        RMotoDir.SetGpioValue(1);
        sleep(2);
        LMotoDir.SetGpioValue(0);
        RMotoDir.SetGpioValue(0);
        sleep(2);
    }
}
```

电机测试程序中，电机 PWM 使用的是设备文件的方式配置并输出 PWM，大家也可以更换为配置寄存器的方式。使用配置寄存器的方式控制电机的方向，在 while 循环里每两秒转换一次电机方向。

八、其他功能测试

```
// CameraDemo();            // 测试摄像头功能
// AdcFunDemo();            // 测试 ADC 功能
// TFTDemo();               // 测试 TFT 屏幕功能
// Demo_IIC_Gyro();         // 测试陀螺仪功能
// Demo_I2C_Gyro();         // 测试陀螺仪驱动功能
// Uart_Demo();             // 测试串口功能
// GetTimeDemo();           // 测试时间戳打印功能
// sleepDemo();             // 测试 sleep() 函数 ------------ 以秒为单位延时
// usleepDemo();            // 测试 usleep() 函数 ----------- 以微秒为单位延时
// nanosleepDemo();         // 测试 nanosleep() 函数 -------- 以纳秒为单位延时
// clock_nanosleepDemo();   // 测试 clock_nanosleep() 函数 -- 以纳秒为单位延时
// MotorDemo();             // 测试电机功能
// ServoDemo();             // 测试舵机功能
// GpioDemo();              // 测试久久派22个GPIO翻转功能
```

其他测试函数，与上面介绍的相同，其中遇到想要查看的详细实现，就可以跳转到对应的文件中查看。

九、相关文件与视频教程

极速光电组龙芯赛道技术手册获取下载地址:

https://gitee.com/lq-tech/LoongCore-DashCart

龙芯久久派例程库

https://gitee.com/lq-tech/Loongson_2k300_LIB

20 届智能车竞赛|龙芯久久派系列学习教程：

https://www.bilibili.com/video/BV1bbNRegEAj